



Canaã Delas

DOCUMENTAÇÃO TÉCNICA

Canaã Delas AI

Plataforma de saúde feminina para Canaã dos Carajás —
arquitetura, modelo de domínio, casos de uso e especificação da
camada de Inteligência Artificial.

Documento	Documentação Técnica — Versão 1.0
Produto	Canaã Delas AI (canaa-delas-ai v0.1.0)
Plataforma	Aplicação web responsiva (Next.js 15 · App Router)
Município	Canaã dos Carajás — Pará, Brasil
Data de emissão	05 de agosto de 2026
Classificação	Documento interno de engenharia

Sumário

1. Visão geral do produto

Objetivo, escopo e público-alvo

Proposta de valor

2. Requisitos

Requisitos funcionais

Requisitos não funcionais

3. Arquitetura da solução

Visão em camadas

Stack tecnológico

Organização do código

4. Modelo de domínio

Diagrama de classes — núcleo de saúde

Diagrama de classes — assistente

Dicionário de dados

Decisões de modelagem

5. Casos de uso

Diagrama de casos de uso

Especificação dos casos principais

6. Comportamento e fluxos

Sequência — autenticação

Sequência — conversa com a assistente

Estados — fases do ciclo menstrual

Mapa de navegação

7. Camada de Inteligência Artificial

8. Algoritmos de domínio

9. Segurança, privacidade e conformidade

10. Interfaces de programação

11. Qualidade e verificação

12. Implantação e operação

13. Limitações conhecidas e evolução

14. Glossário

1. Visão geral do produto

1.1 Objetivo, escopo e público-alvo

O **Canaã Delas AI** é uma plataforma digital de saúde feminina concebida para a realidade das mulheres de Canaã dos Carajás (PA). O sistema reúne, em um único ambiente autenticado, o acompanhamento do ciclo menstrual, da gestação, do bem-estar emocional e da prevenção em saúde, somados a uma camada de Inteligência Artificial que personaliza orientações a partir dos registros da própria usuária.

DIMENSÃO	DEFINIÇÃO
Público primário	Mulheres em idade reprodutiva residentes em Canaã dos Carajás.
Público secundário	Gestão pública municipal (dados agregados anonimizados) e parceiros de saúde.
Escopo desta versão	Aplicação web responsiva, multiusuária, com persistência local e assistente conversacional.
Fora de escopo	Aplicativo nativo, prontuário eletrônico, telemedicina, prescrição e agendamento no SUS.

Natureza do produto. O sistema é uma ferramenta de *autoconhecimento e organização pessoal em saúde*. Não é dispositivo médico, não realiza diagnóstico e não substitui avaliação por profissional de saúde. Essa restrição é aplicada em nível de produto (avisos nas telas) e de engenharia (instruções explícitas no *prompt* da assistente).

1.2 Proposta de valor

EIXO	ENTREGA
Autoconhecimento	Previsão de ciclo, fase atual, janela fértil e estatísticas de regularidade.
Prevenção	Lembretes de exames, guia de autoexame das mamas e checklist por faixa etária.
Personalização por IA	Assistente que conhece os registros da usuária e acumula memória de longo prazo.
Vínculo com o território	Serviços públicos do SUS, campanhas municipais e rede de apoio local.
Autonomia sobre os dados	Exportação integral, exclusão de conta e gestão da memória da assistente.

2. Requisitos

2.1 Requisitos funcionais

Situação de implementação na versão 1.0:

IMPLEMENTADO

PARCIAL / PLANEJADO

REQUER SERVIÇO EXTERNO

ID	REQUISITO	SITUAÇÃO
RF-01	Cadastro e autenticação por e-mail e senha	IMPLEMENTADO
RF-02	Onboarding com definição de objetivo pessoal	IMPLEMENTADO
RF-03	Registro de ciclos menstruais com fluxo e sintomas	IMPLEMENTADO
RF-04	Calendário visual com menstruação, previsão, janela fértil e ovulação	IMPLEMENTADO
RF-05	Cálculo de fase do ciclo e estatísticas de regularidade	IMPLEMENTADO
RF-06	Acompanhamento gestacional com idade gestacional, DPP e marcos	IMPLEMENTADO
RF-07	Registro de humor e check-in diário (energia, sono, dor, sintomas)	IMPLEMENTADO
RF-08	Lembretes de exames, consultas e medicação	IMPLEMENTADO
RF-09	Registro de medidas de saúde (peso, pressão, glicemia, temperatura basal)	IMPLEMENTADO
RF-10	Controle diário de anticoncepcional com adesão e sequência	IMPLEMENTADO
RF-11	Guia e registro do autoexame das mamas	IMPLEMENTADO
RF-12	Biblioteca de conteúdo educativo	IMPLEMENTADO
RF-13	Diretório de serviços públicos e rede de apoio	IMPLEMENTADO
RF-14	Insights automáticos derivados dos registros	IMPLEMENTADO
RF-15	Relatório consolidado para consulta médica (imprimível)	IMPLEMENTADO
RF-16	Metas, sequências e conquistas de engajamento	IMPLEMENTADO
RF-17	Assistente conversacional com contexto isolado por conversa	IMPLEMENTADO
RF-18	Memória de longo prazo da assistente, visível e removível	IMPLEMENTADO
RF-19	Exportação integral e exclusão definitiva dos dados	IMPLEMENTADO
RF-20	PIN de bloqueio do aplicativo	PARCIAL
RF-21	Notificações <i>push</i> de lembretes	PLANEJADO
RF-22	Assinatura Premium com pagamento	EXTERNO
RF-23	<i>Marketplace</i> e rede de parceiros	EXTERNO
RF-24	Painel de indicadores para gestão pública	PLANEJADO

2.2 Requisitos não funcionais

ID	CATEGORIA	REQUISITO
RNF-01	Segurança	Senhas e PIN armazenados exclusivamente como <i>hash</i> bcrypt (fator 10).
RNF-02	Segurança	Toda leitura e escrita filtrada pelo <code>userId</code> da sessão — isolamento entre usuárias.
RNF-03	Segurança	Rotas sob <code>/panel</code> protegidas por <i>middleware</i> no <i>edge</i> .
RNF-04	Privacidade	Direito de acesso e eliminação atendidos por autoatendimento (LGPD).
RNF-05	Usabilidade	Interface responsiva, em português do Brasil, com navegação inferior no <i>mobile</i> .
RNF-06	Acessibilidade	HTML semântico, rótulos associados a campos e foco visível em elementos interativos.
RNF-07	Manutenibilidade	TypeScript em modo <i>strict</i> ; validação de entrada centralizada em esquemas Zod.
RNF-08	Desempenho	Renderização no servidor por padrão; <i>JavaScript</i> compartilhado de ~103 kB.
RNF-09	Portabilidade	Modelagem sem recursos exclusivos do SQLite, permitindo migração a PostgreSQL.
RNF-10	Resiliência	Falha na camada de IA não impede o uso das demais funcionalidades.
RNF-11	Segurança clínica	Vedação explícita a diagnóstico e prescrição; encaminhamento a serviço de saúde.

3. Arquitetura da solução

3.1 Visão em camadas

A aplicação adota arquitetura monolítica modular sobre o *App Router* do Next.js. A renderização ocorre predominantemente no servidor (*React Server Components*); as mutações trafegam por *Server Actions*, e apenas o *streaming* da assistente utiliza um *Route Handler* HTTP convencional, por exigir resposta progressiva.

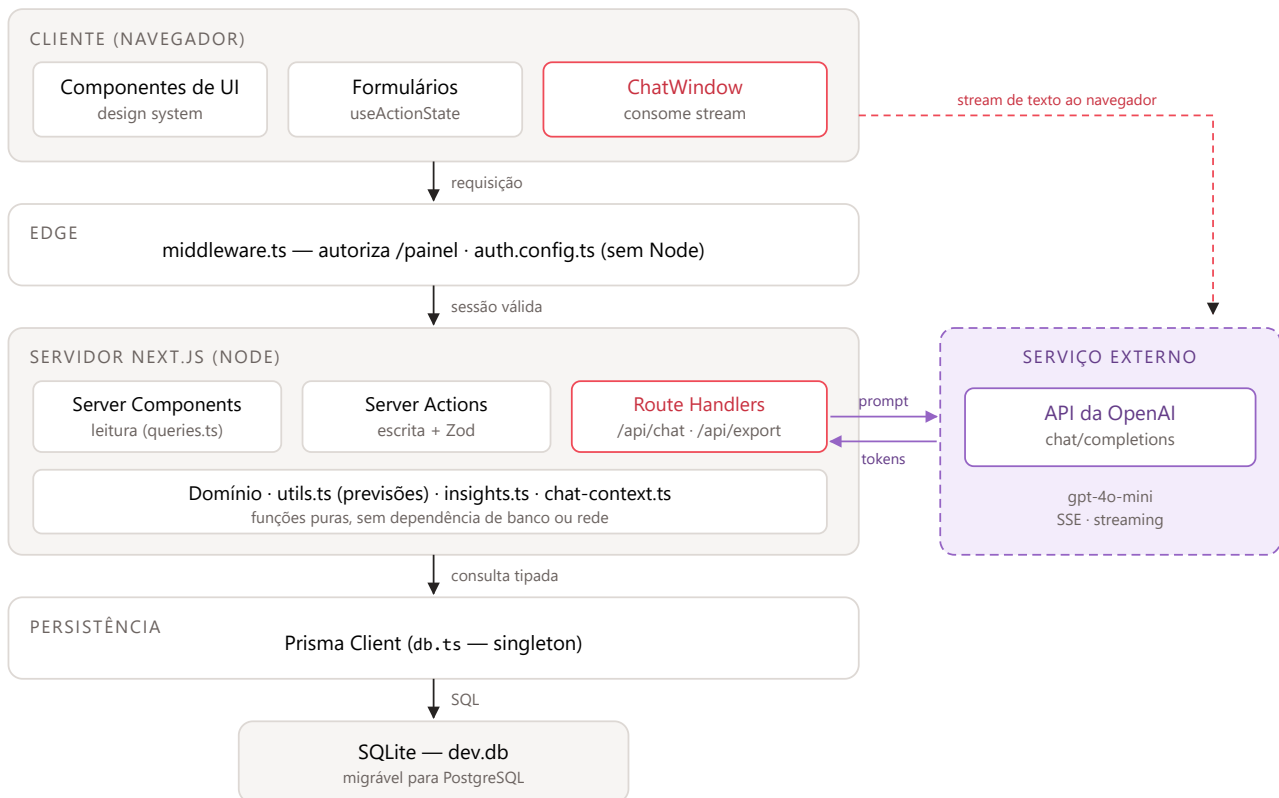


Figura 1 — Arquitetura em camadas. A requisição atravessa o *middleware* antes de alcançar o servidor; o domínio é isolado em funções puras. A API da OpenAI é o único serviço externo, acionado exclusivamente pelo *Route Handler* `/api/chat`, cuja resposta retorna ao navegador em *streaming*.

3.2 Stack tecnológico

CAMADA	TECNOLOGIA	VERSÃO	JUSTIFICATIVA
Runtime	Node.js	24.12	Suporte nativo a <i>streams</i> web e <code>fetch</code> .
Framework	Next.js (App Router)	15.5.22	Renderização no servidor, <i>Server Actions</i> e roteamento por arquivos.
Interface	React	19.0	<code>useActionState</code> e <code>useFormStatus</code> para formulários progressivos.
Linguagem	TypeScript (strict)	5.7	Contratos verificados em tempo de compilação.
Estilo	Tailwind CSS	4.0	<i>Design tokens</i> em CSS nativo via <code>@theme</code> .
Persistência	Prisma ORM + SQLite	6.x	Cliente tipado gerado a partir do <i>schema</i> .
Autenticação	Auth.js (NextAuth) + <code>bcryptjs</code>	5.0-beta	Sessão por JWT compatível com <i>edge</i> .
Validação	Zod	3.24	Esquema único como fonte de verdade e de tipos.
IA	API da OpenAI	gpt-4o-mini	Acesso via <code>fetch</code> , sem SDK, reduzindo dependências.

3.3 Organização do código

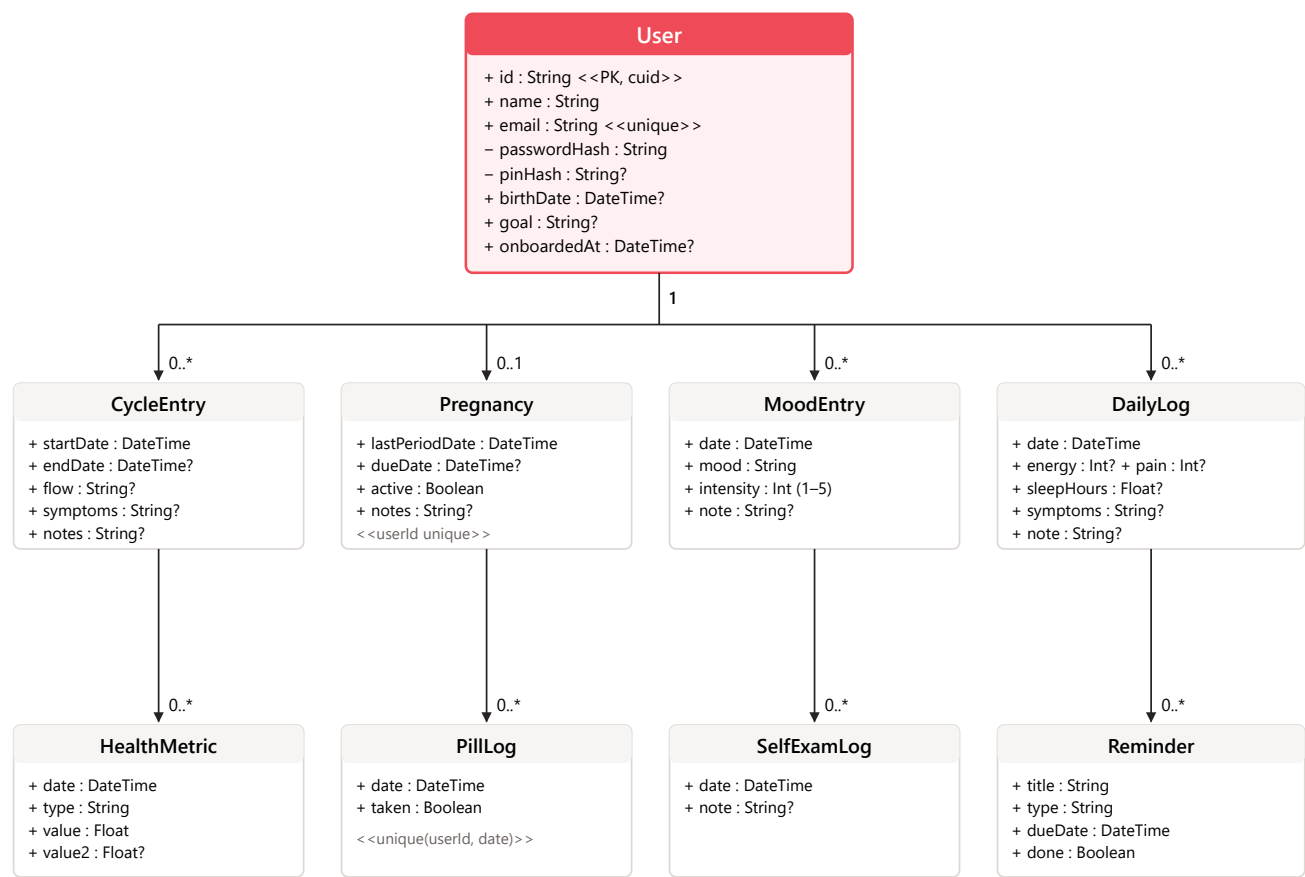
src/	
└─ app/	Roteamento (App Router)
│ └─ (auth)/	Grupo público: login, cadastro
│ └─ (dashboard)/painel/	Grupo autenticado: 20 telas
│ └─ api/	Route Handlers
│ │ └─ auth/[...nextauth]/	Endpoints do Auth.js
│ │ └─ chat/	Streaming da assistente
│ │ └─ export/	Exportação de dados (LGPD)
│ │ └─ logout/	Encerramento de sessão órfã
│ └─ comecar/	Onboarding
│ └─ layout.tsx	Layout raiz, fontes e metadados
│ └─ globals.css	Design tokens (@theme)
└─ components/	
│ └─ ui/	Design system (Button, Card, Icon...)
│ └─ layout/	Sidebar, cabeçalho, marca
│ └─ auth/	Formulários de autenticação
│ └─ features/	Formulários e widgets de domínio
└─ server/	
│ └─ actions/	Mutações (Server Actions) por domínio
│ └─ queries.ts	Leituras reutilizáveis
└─ lib/	
│ └─ auth.ts / session.ts	Autenticação e guarda de sessão
│ └─ db.ts	Prisma Client (singleton)
│ └─ utils.ts	Datas, previsões, estatísticas
│ └─ insights.ts	Geração de insights
│ └─ openai.ts	Cliente da OpenAI + streaming
│ └─ chat-context.ts	Prompt de sistema e memória
│ └─ validations.ts	Esquemas Zod
│ └─ constants.ts	Enumerações de domínio
└─ auth.config.ts	Configuração compatível com edge
└─ middleware.ts	Proteção de rotas

Separação leitura / escrita. Consultas residem em `server/queries.ts` e são chamadas diretamente por *Server Components*; mutações residem em `server/actions/`, sempre validando a entrada com Zod e invalidando o cache com `revalidatePath`. A divisão mantém as funções de leitura reutilizáveis e testáveis sem o ciclo de vida de uma ação.

4. Modelo de domínio

O domínio é composto por doze entidades persistidas. `User` é a raiz de agregação: todas as demais entidades pertencem a exatamente uma usuária e são eliminadas em cascata com ela (`onDelete: Cascade`). Para legibilidade, o modelo é apresentado em duas figuras — o núcleo de saúde e a camada da assistente.

4.1 Diagrama de classes — núcleo de saúde



Todas as associações partem de `User` com exclusão em cascata (`onDelete: Cascade`).
Convenção: (+) atributo de domínio · (-) atributo sensível, nunca exposto ao cliente.
Campos `id`, `userId`, `createdAt` e `updatedAt` omitidos das caixas — presentes em todas as entidades.

Figura 2 — Diagrama de classes do núcleo de saúde. Oito entidades de registro pendem de `User`. `Pregnancy` é a única associação *um-para-zero-ou-um*, garantida por restrição de unicidade em `userId`; `PillLog` impede duplicidade diária por unicidade composta.

4.2 Diagrama de classes — assistente

A camada conversacional distingue dois tipos de memória: o **histórico**, restrito a uma conversa (`ChatMessage` pertence a `Conversation`), e a **memória de longo prazo** (`ChatMemory`), associada diretamente à usuária e, portanto, compartilhada por todas as conversas. Essa distinção é o mecanismo que permite conversas com contextos isolados e, ao mesmo tempo, uma assistente que reconhece a usuária.

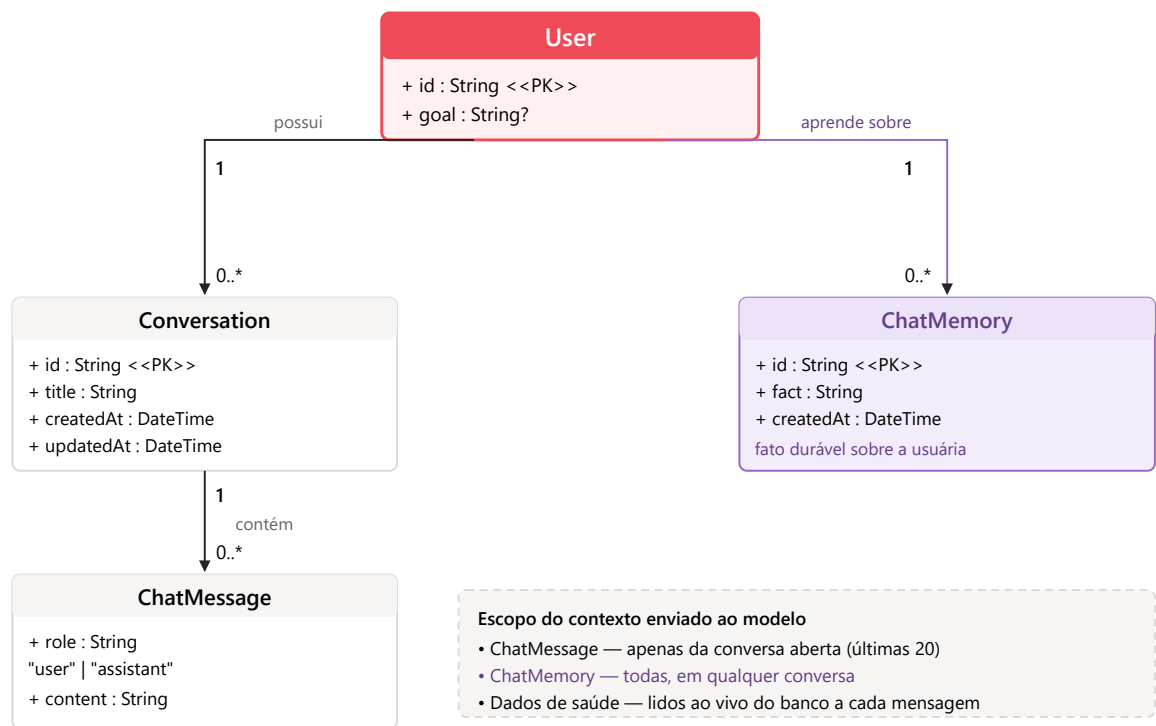


Figura 3 — Diagrama de classes da assistente. O isolamento entre conversas decorre de `ChatMessage` pender de `Conversation`; a continuidade entre elas decorre de `ChatMemory` pender diretamente de `User`.

4.3 Dicionário de dados

ENTIDADE	REGISTROS	FINALIDADE E OBSERVAÇÕES
User	1 por conta	Identidade, credenciais (hash), objetivo e marco de onboarding.
CycleEntry	0..n	Ciclo menstrual. Base de todas as previsões. Índice em (userId, startDate).
Pregnancy	0..1	Gestação ativa. lastPeriodDate (DUM) origina idade gestacional e DPP.
MoodEntry	0..n	Humor com intensidade de 1 a 5.
DailyLog	0..n	Check-in diário: energia, dor, sono e sintomas.
HealthMetric	0..n	Série temporal de medidas. value2 comporta a pressão diastólica.
PillLog	0..n	Adesão ao anticoncepcional; unicidade composta evita duplicidade diária.
SelfExamLog	0..n	Registro do autoexame das mamas.
Reminder	0..n	Exames, consultas e medicação, com estado de conclusão.
Conversation	0..n	Sessão de diálogo; título derivado da primeira mensagem.
ChatMessage	0..n	Turno de diálogo, ordenado por createdAt.
ChatMemory	0..n	Fato durável sobre a usuária, reinjetado em todas as conversas.

4.4 Decisões de modelagem

DECISÃO	FUNDAMENTO E CONSEQUÊNCIA
Enumerações como String	O SQLite não possui tipo <i>enum</i> . Os valores admissíveis residem em <code>lib/constants.ts</code> e são impostos por Zod na fronteira de escrita. Preserva a portabilidade a PostgreSQL sem migração de tipos.
Listas em campo textual	<code>symptoms</code> armazena valores separados por vírgula em vez de tabela associativa. Escolha deliberada: o dado é lido em bloco e nunca consultado por item. Caso surja necessidade de filtro por sintoma, exige normalização.
Exclusão em cascata	Todas as entidades declaram <code>onDelete: Cascade</code> , de modo que a eliminação da conta remove integralmente os dados — requisito de LGPD atendido pelo próprio esquema.
Ausência de <i>soft delete</i>	Registros de saúde excluídos são removidos fisicamente. Privilegia-se o direito à eliminação sobre a auditabilidade.
Memória separada do histórico	<code>ChatMemory</code> associa-se a <code>User</code> , não a <code>Conversation</code> , viabilizando continuidade entre conversas sem transportar histórico integral no <i>prompt</i> .

5. Casos de uso

5.1 Diagrama de casos de uso

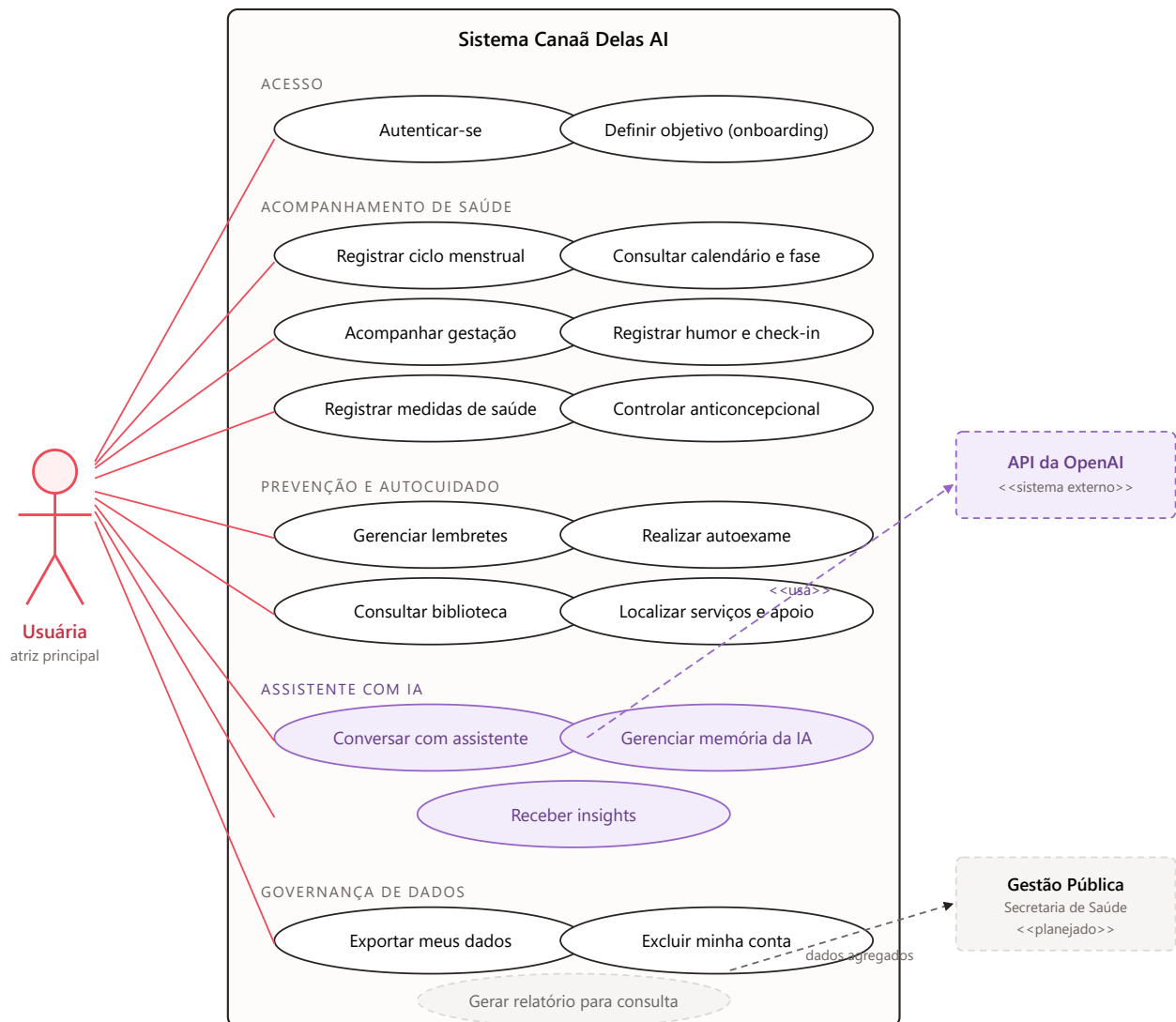


Figura 4 — Diagrama de casos de uso. A usuária é a única atriz com acesso direto ao sistema. A API da OpenAI participa apenas dos casos de uso da assistente, em relação de dependência (<<usa>>). Elementos tracejados correspondem a escopo planejado.

5.2 Especificação dos casos principais

UC-01 — Registrar ciclo menstrual

Atriz	Usuária autenticada
Objetivo	Registrar o início (e opcionalmente o término) de um ciclo, com fluxo e sintomas.
Pré-condição	Sessão válida e onboarding concluído.
Pós-condição	<code>CycleEntry</code> persistido; previsões, fase e calendário recalculados.
Fluxo principal	1. A usuária acessa <code>/painel/ciclo</code>. 2. Informa a data de início; opcionalmente fim, fluxo, sintomas e observações. 3. Submete o formulário, acionando <code>addCycleAction</code>. 4. O sistema valida a entrada com <code>cycleSchema</code> (Zod). 5. Persiste o registro vinculado ao <code>userId</code> da sessão. 6. Invalida o cache das rotas <code>/painel</code> e <code>/painel/ciclo</code>. 7. A tela reapresenta calendário, fase atual e estatísticas atualizados.
Fluxo alternativo	A4a. Validação falha: retorna erros por campo, preservando o formulário.
Regras	Intervalos entre 15 e 60 dias são considerados válidos para cálculo de média; os demais são descartados como discrepantes.

UC-02 — Conversar com a assistente

Atrizes	Usuária autenticada; API da OpenAI (ator externo)
Objetivo	Obter orientação personalizada em linguagem natural, com contexto isolado por conversa.
Pré-condição	Sessão válida; <code>OPENAI_API_KEY</code> configurada; conversa pertencente à usuária.
Pós-condição	Par de mensagens persistido; eventuais fatos duráveis acrescidos a <code>ChatMemory</code> .
Fluxo principal	1. A usuária cria ou seleciona uma conversa. 2. Envia a mensagem; o cliente requisita <code>POST /api/chat</code>. 3. O sistema verifica a sessão e a titularidade da conversa. 4. Recupera as últimas 20 mensagens <i>daquela</i> conversa. 5. Monta o <i>prompt</i> de sistema com dados de saúde ao vivo e a memória acumulada. 6. Persiste a mensagem da usuária e requisita a OpenAI em modo <i>streaming</i>. 7. Repassa os fragmentos ao navegador, que os exibe progressivamente. 8. Ao concluir, persiste a resposta e executa a extração de memória.
Fluxos de exceção	3a. Sessão ausente: <code>401</code>. 3b. Conversa de outra usuária: <code>404</code>. 6a. Chave não configurada: <code>503</code> com orientação de configuração. 8a. Falha na extração de memória: registrada e ignorada — não afeta a resposta entregue.
Regras	Vedados diagnóstico e prescrição. Assuntos graves ensejam encaminhamento a serviço de saúde.

UC-03 — Exportar e excluir dados pessoais

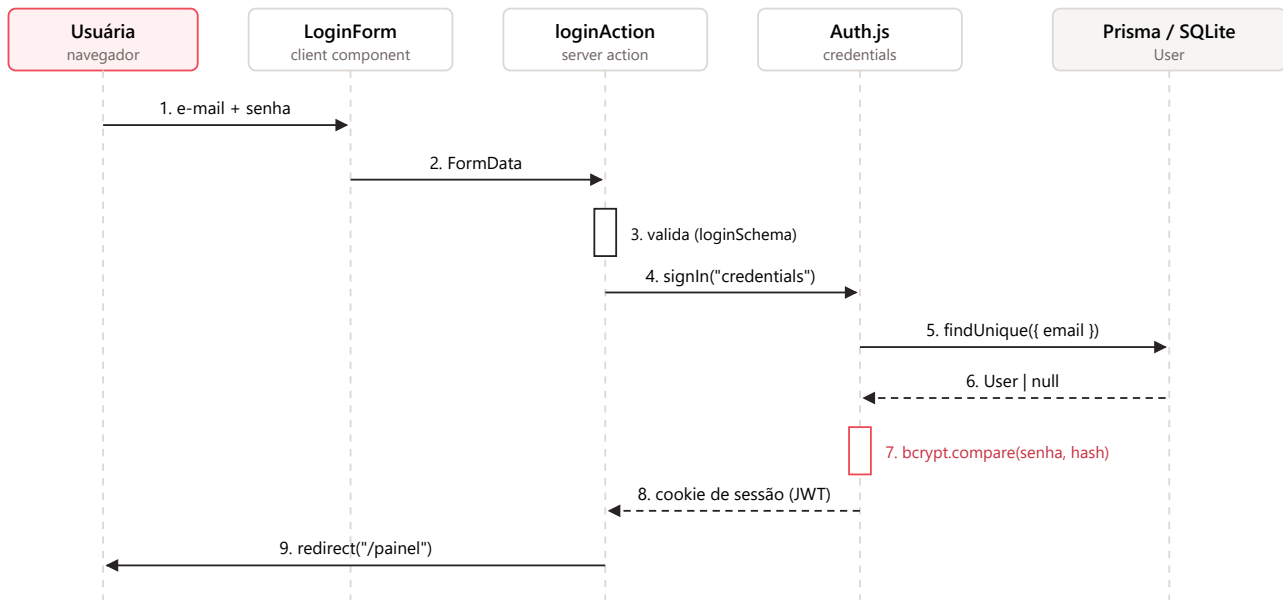
Atriz	Usuária autenticada
Objetivo	Exercer os direitos de acesso e de eliminação previstos na LGPD.
Fluxo — exportação	1. A usuária acessa <code>/painel/configuracoes</code> e solicita a exportação. 2. O sistema requisita <code>GET /api/export</code>. 3. Agrega todas as entidades da usuária, suprimindo <code>passwordHash</code> e <code>pinHash</code>. 4. Responde com arquivo JSON como anexo.
Fluxo — exclusão	1. A usuária confirma a exclusão em diálogo explícito. 2. <code>deleteAccountAction</code> remove o registro de <code>User</code>. 3. A cascata do banco elimina todas as entidades dependentes. 4. A sessão é encerrada e a usuária redirecionada à página inicial.
Pós-condição	Nenhum dado da usuária permanece no banco. A operação é irreversível.

UC-04 — Acompanhar gestação

Atriz	Usuária autenticada
Objetivo	Acompanhar a evolução da gestação a partir da data da última menstruação.
Fluxo principal	1. A usuária informa a DUM em <code>/painel/gestacao</code>. 2. <code>upsertPregnancyAction</code> valida e calcula a DPP (DUM + 280 dias). 3. Persiste em <code>Pregnancy</code> por <i>upsert</i> — uma gestação ativa por usuária. 4. O sistema apresenta idade gestacional, trimestre, percentual de progresso e marco da semana.
Regras	Trimestres: 1. ^o até a 12. ^a semana; 2. ^o da 13. ^a à 26. ^a ; 3. ^o a partir da 27. ^a .

6. Comportamento e fluxos

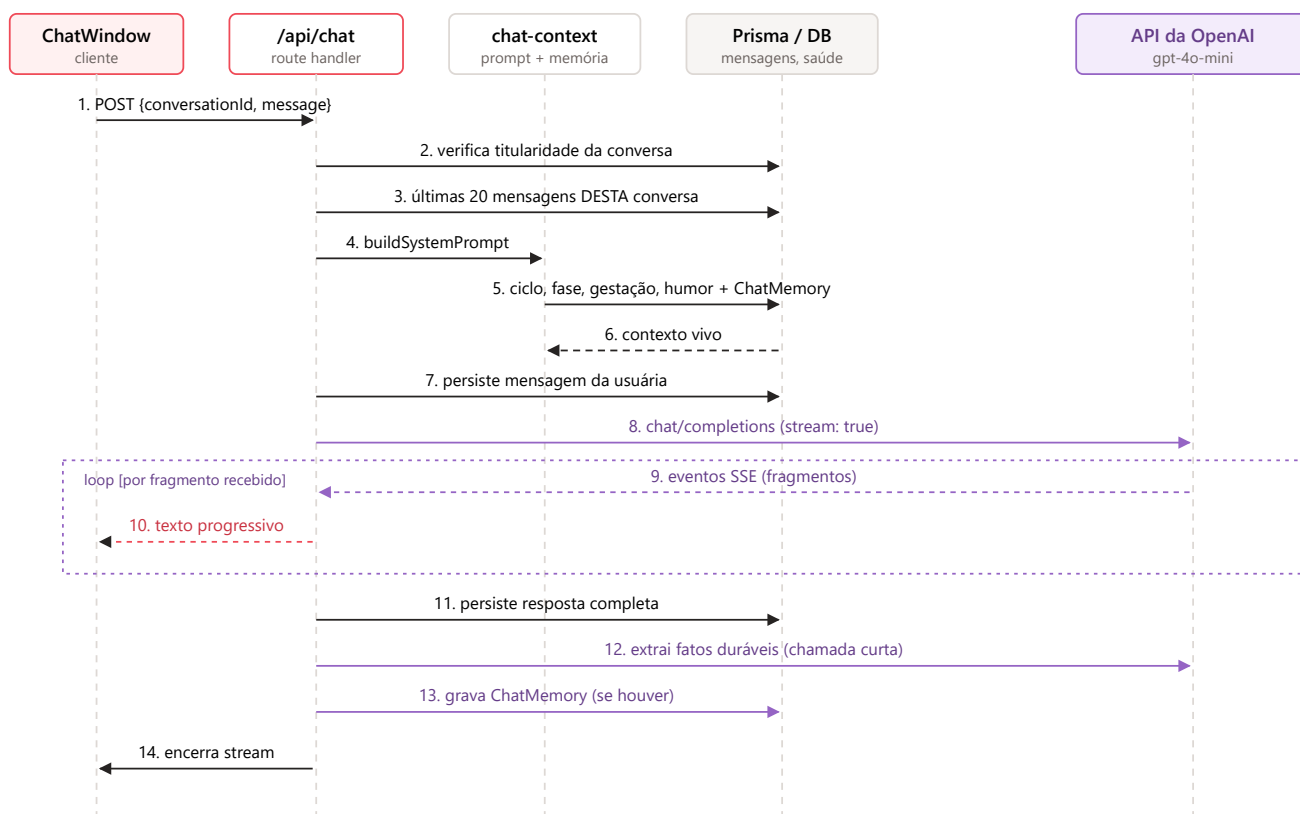
6.1 Sequência — autenticação



Credenciais inválidas (passo 7 falso) retornam mensagem genérica — não se revela se o e-mail existe.

Figura 5 — Sequência de autenticação. A senha é comparada apenas em memória, no servidor, contra o *hash* bcrypt; a sessão trafega como JWT em *cookie*, formato legível pelo *middleware* no *edge* sem acesso ao banco.

6.2 Sequência — conversa com a assistente

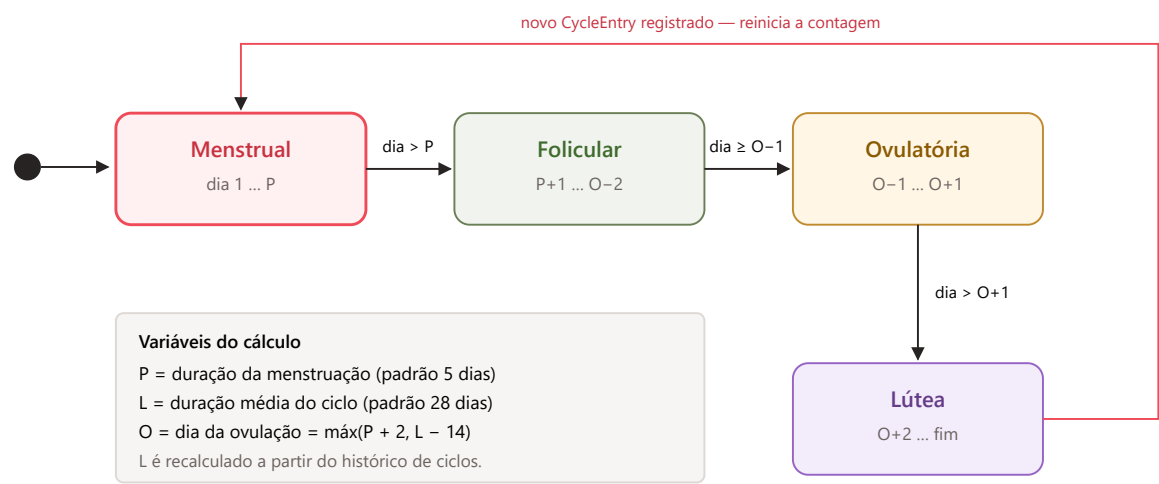


Passos 12–13 ocorrem antes do encerramento do stream, garantindo a gravação da memória; a resposta já está visível na tela. Falha em 12 ou 13 é registrada e ignorada — a memória é um recurso acessório (best-effort).

Figura 6 — Sequência da conversa com a assistente. O isolamento de contexto ocorre no passo 3 (somente mensagens da conversa corrente), enquanto a continuidade entre conversas provém do passo 5, que injeta a memória acumulada e os dados de saúde lidos ao vivo.

6.3 Estados — fases do ciclo menstrual

A fase do ciclo é derivada, não armazenada: `cyclePhase()` a calcula a cada leitura a partir do dia corrente, da duração média do ciclo e da duração da menstruação. A transição é, portanto, função exclusiva do tempo decorrido desde o último registro de início.



Fase derivada em tempo de leitura — não há coluna de estado no banco. Sem registros, não há fase definida.

Figura 7 — Estados das fases do ciclo. As transições dependem apenas do dia decorrido; o registro de um novo ciclo reinicia a máquina de estados na fase menstrual.

6.4 Mapa de navegação

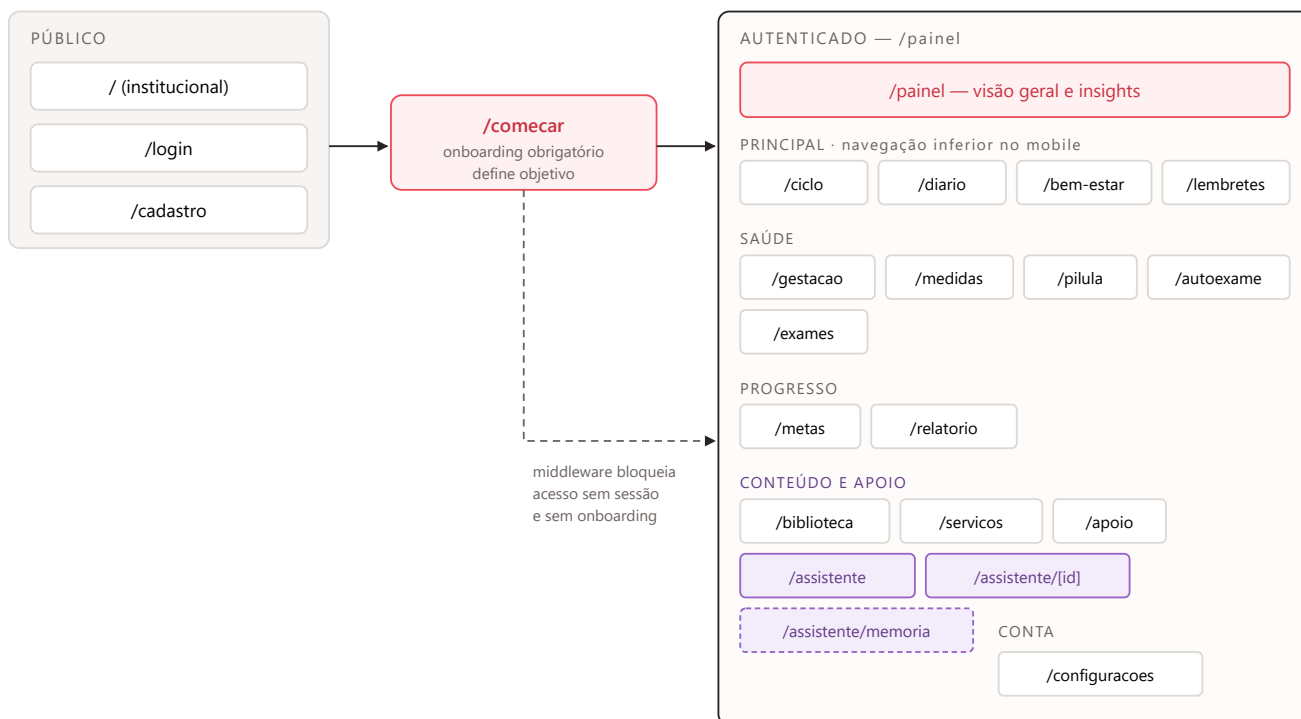


Figura 8 — Mapa de navegação. Vinte telas autenticadas organizadas em cinco grupos. Nenhuma rota sob `/painel` é alcançável sem sessão válida e onboarding concluído.

7. Camada de Inteligência Artificial

A assistente é o diferencial funcional do produto. Seu comportamento resulta da composição de três fontes de contexto, montadas a cada mensagem enviada.

FONTE DE CONTEXTO	ESCOPO	CONTEÚDO E ORIGEM
Persona e limites	Fixo	Instruções de tom, idioma e vedações clínicas. Definido em <code>chat-context.ts</code> .
Dados de saúde	Lido ao vivo	Dia e fase do ciclo, previsões, estatísticas, gestação, humores e sintomas recorrentes. Consultado no banco a cada mensagem — nunca fica obsoleto.
Memória de longo prazo	Acumulado	Até 30 fatos duráveis de ChatMemory, compartilhados por todas as conversas.
Histórico da conversa	Isolado	Últimas 20 mensagens <i>daquela</i> conversa — o que garante contextos independentes.

7.1 Ciclo de aprendizado

Concluída cada resposta, uma segunda chamada de baixo custo analisa a troca e extrai *apenas fatos duráveis*. A distinção entre durável e passageiro é decisiva: sem ela, a memória se contamina com dados que envelhecem mal.

MEMORIZADO (DURÁVEL)	DESCARTADO (PASSAGEIRO OU DERIVÁVEL)
Condições e diagnósticos relatados	Dia ou fase atual do ciclo
Alergias e medicação de uso contínuo	Humor e sintomas do dia
Método contraceptivo em uso	Horas de sono de uma noite
Histórico gestacional e cirúrgico	Datas previstas
Preferências e contexto de vida	Explicações dadas pela própria assistente
Padrões recorrentes relatados	Perguntas, saudações e agradecimentos

Defeito identificado e corrigido durante a implementação. A primeira versão do extrator memorizou o registro *“Ela está no dia 37 do seu ciclo”*. Trata-se de informação passageira: no dia seguinte estaria incorreta e passaria a contaminar todas as respostas subsequentes. O *prompt* de extração foi então reescrito com exemplos negativos explícitos e a diretriz de responder **NADA** em caso de dúvida. Após a correção, a mesma troca passou a produzir os fatos *“Tem endometriose”*, *“Costuma ter cólica forte no primeiro dia”* e *“Usa DIU de cobre”*, descartando o dado do ciclo.

7.2 Segurança clínica

SALVAGUARDA	IMPLEMENTAÇÃO
Vedação a diagnóstico	Instrução explícita no <i>prompt</i> de sistema.
Vedação a prescrição	Proibição de indicar medicamento ou dosagem.
Encaminhamento	Orientação a procurar UBS ou profissional diante de sinais de alerta.
Emergência	Telefones 192 (SAMU) e 180 (Central de Atendimento à Mulher).
Vedação à invenção	Instrução para admitir desconhecimento e não inferir dados de saúde.
Aviso na interface	Texto permanente sob o campo de mensagem.

7.3 Parâmetros de operação

PARÂMETRO	VALOR
Modelo	<code>gpt-4o-mini</code> (configurável por <code>OPENAI_MODEL</code>)
Transporte	<code>fetch</code> nativo sobre <code>chat/completions</code> , sem SDK
Streaming	<i>Server-Sent Events</i> convertidos em <code>ReadableStream</code> de texto
Temperatura	0,7 no diálogo · 0,3 na extração de memória
Limite de resposta	800 <i>tokens</i> no diálogo · 200 na extração
Janela de histórico	20 mensagens · 30 fatos de memória
Degradação	Ausência de chave resulta em HTTP 503 com orientação; o restante do sistema segue operante

8. Algoritmos de domínio

As regras de cálculo residem em `lib/Utils.ts` como funções puras, sem dependência de banco ou rede — condição que as torna determinísticas e verificáveis isoladamente.

8.1 Previsão de ciclo — `predictCycle()`

```
Entrada: lista de datas de início de ciclos
1. Ordenar as datas em ordem decrescente; seja U a mais recente
2. Calcular os intervalos entre inícios consecutivos
3. Descartar intervalos fora de [21, 45] dias (dados incompletos)
4. L ← média dos intervalos restantes, arredondada
   (na ausência de histórico suficiente, L ← 28)
5. Próxima menstruação      ← U + L
6. Ovulação estimada        ← (U + L) - 14
7. Janela fértil             ← [ovulação - 5, ovulação + 1]
8. Dia atual do ciclo        ← (hoje - U) + 1
```

O ancoramento da ovulação em quatorze dias *antes* da próxima menstruação — e não após o início da anterior — segue a constatação clínica de que a fase lútea é mais estável que a folicular, o que reduz o erro em ciclos de duração atípica.

8.2 Estatísticas de regularidade — `cycleStats()`

```
Entrada: lista de datas de início
1. Calcular intervalos consecutivos; descartar valores fora de [15, 60]
2. média, mínimo, máximo ← sobre os intervalos válidos
3. variação ← máximo - mínimo
4. Classificação:
   variação ≤ 4 dias → "regular"
   variação > 4 dias → "irregular"
   menos de 2 intervalos → "insuficiente"
```

8.3 Progresso gestacional — `pregnancyProgress()`

```
Entrada: DUM (data da última menstruação)
1. dias      ← hoje - DUM
2. semanas   ← [dias ÷ 7]      resto ← dias mod 7
3. DPP       ← DUM + 280 dias  (40 semanas - regra de Naegele)
4. trimestre ← semanas < 13 → 1 ; semanas < 27 → 2 ; senão 3
5. progresso % ← mín(100, [dias ÷ 280 × 100])
```

8.4 Geração de insights — `generateInsights()`

Regras heurísticas determinísticas, sem uso de modelo de linguagem. Retorna no máximo cinco itens, por ordem de relevância.

CONDIÇÃO AVALIADA	INSIGHT PRODUZIDO
Janela fértil em até 3 dias	Antecipação da janela fértil com a data
Caso contrário, havendo previsão	Data prevista da próxima menstruação
Fase lútea corrente	Alerta antecipado de sintomas pré-menstruais
Histórico com média disponível	Regularidade e amplitude de variação
Sintoma repetido 2 ou mais vezes	Sintoma predominante e sua frequência
3 ou mais registros de sono	Média de horas de sono
3 ou mais registros de humor	Humor predominante

8.5 Sequência de adesão — `currentStreak()`

Entrada: lista de datas de registro

1. Normalizar as datas ao início do dia; formar conjunto de chaves
2. Fixar o cursor em hoje, se houver registro; senão em ontem; não havendo nenhum dos dois, retornar 0
3. Enquanto o cursor existir no conjunto: incrementar contagem; recuar um dia
4. Retornar a contagem

A tolerância ao dia anterior é intencional: preserva a sequência de quem ainda não registrou o dia corrente, evitando penalizar a usuária pelo horário de acesso.

9. Segurança, privacidade e conformidade

9.1 Autenticação e sessão

A configuração do Auth.js é deliberadamente cindida em dois módulos, exigência do *Edge Runtime*, que não executa Prisma nem bcrypt:

MÓDULO	AMBIENTE	RESPONSABILIDADE
auth.config.ts	Edge	Autorização de rotas e <i>callbacks</i> de sessão. Sem dependências de Node.
lib/auth.ts	Node	<i>Provider</i> de credenciais, consulta ao banco e comparação bcrypt.

9.2 Controles implementados

CONTROLE	IMPLEMENTAÇÃO
Armazenamento de senha	bcrypt com fator de custo 10. A senha em claro nunca é persistida nem registrada em <i>log</i> .
Isolamento entre usuárias	Toda consulta e mutação filtra por <i>userId</i> da sessão. As exclusões empregam <code>deleteMany({ where: { id, userId } })</code> , tornando inócua a manipulação de identificadores.
Proteção de rotas	<i>Middleware</i> nega acesso a <code>/painel</code> sem sessão; o layout autenticado impõe adicionalmente a conclusão do onboarding.
Validação de entrada	Esquemas <i>Zod</i> na fronteira de toda ação e <i>route handler</i> .
Mensagens de erro	Falha de autenticação retorna texto genérico, sem revelar a existência do e-mail.
Supressão em exportações	<code>passwordHash</code> e <code>pinHash</code> são removidos do JSON de exportação.
Sessão órfã	Sessão apontando para usuária inexistente aciona encerramento automático em <code>/api/logout</code> , em lugar de falha de execução.
Segredos	<code>AUTH_SECRET</code> e <code>OPENAI_API_KEY</code> permanecem exclusivamente no servidor; <code>.env</code> é ignorado pelo controle de versão.

9.3 Aderência à LGPD

DIREITO DO TITULAR (LEI 13.709/2018)	ATENDIMENTO
Acesso aos dados (art. 18, II)	<code>GET /api/export</code> — cópia integral em JSON.
Eliminação (art. 18, VI)	Exclusão de conta com cascata em todas as entidades.
Correção (art. 18, III)	Edição e remoção de registros nas respectivas telas.
Transparência (art. 9.º)	Tela <code>/painel/assistente/memoria</code> expõe integralmente o que a IA armazenou.
Minimização (art. 6.º, III)	Coleta restrita ao necessário; data de nascimento é facultativa.

Ressalvas de segurança para produção. A versão atual é adequada a desenvolvimento e demonstração.

Antes da operação com dados reais são requisitos: (i) migração para PostgreSQL com criptografia em repouso; (ii) tráfego exclusivamente sob HTTPS; (iii) limitação de taxa nas rotas de autenticação e de chat; (iv) verificação de e-mail e recuperação de senha; (v) rotação de `AUTH_SECRET` ; (vi) política de privacidade e termo de consentimento; (vii) conclusão da tela de bloqueio por PIN; (viii) avaliação do envio de dados de saúde a serviço de terceiro (OpenAI) à luz do consentimento obtido.

10. Interfaces de programação

10.1 Route Handlers

MÉTODO	ROTA	DESCRIÇÃO E RESPOSTAS
POST	/api/chat	Diálogo com a assistente. Corpo: { conversationId, message }. Respostas: 200 (stream de texto), 400 (entrada inválida), 401 (sem sessão), 404 (conversa de outra usuária), 503 (chave ausente).
GET	/api/export	Exportação integral em JSON, como anexo. 200 · 401 · 404.
GET	/api/logout	Encerra a sessão e redireciona a /login. Usado na recuperação de sessão órfã.
GET/POST	/api/auth/[...nextauth]	Endpoints do Auth.js (CSRF, callback de credenciais, sessão).

10.2 Server Actions

Vinte e seis ações agrupadas por domínio em `src/server/actions/`. Todas exigem sessão válida (`requireUser()`), validam a entrada e invalidam o cache das rotas afetadas.

MÓDULO	AÇÕES
auth.ts	registerAction, loginAction, logoutAction
onboarding.ts	completeOnboardingAction
cycle.ts	addCycleAction, deleteCycleAction
pregnancy.ts	upsertPregnancyAction, endPregnancyAction
mood.ts	addMoodAction, deleteMoodAction
daily.ts	addDailyLogAction, deleteDailyLogAction
reminders.ts	addReminderAction, toggleReminderAction, deleteReminderAction
health.ts	addMetricAction, deleteMetricAction, togglePillTodayAction, logSelfExamAction
chat.ts	createConversationAction, deleteConversationAction, clearMemoriesAction
settings.ts	updateGoalAction, setPinAction, removePinAction, deleteAccountAction

10.3 Variáveis de ambiente

VARIÁVEL	OBRIGATÓRIA	FINALIDADE
DATABASE_URL	Sim	Conexão do Prisma. Em desenvolvimento, <code>file:./dev.db</code> .
AUTH_SECRET	Sim	Assinatura do JWT de sessão. Deve ser rotacionada em produção.
AUTH_URL	Sim	URL base para os <i>callbacks</i> do Auth.js.
OPENAI_API_KEY	Para o chat	Credencial da OpenAI. Variáveis reais do ambiente têm precedência sobre <code>.env</code> .
OPENAI_MODEL	Não	Modelo a utilizar. Padrão: <code>gpt-4o-mini</code> .

11. Qualidade e verificação

11.1 Verificações automatizadas

VERIFICAÇÃO	COMANDO	RESULTADO
Tipagem estática	<code>npm run typecheck</code>	Sem erros
Análise estática	<code>npm run lint</code>	Sem advertências
<i>Build</i> de produção	<code>npm run build</code>	Concluído — 30 rotas

11.2 Verificação funcional executada

A camada de IA foi exercitada contra a API real da OpenAI, e não apenas simulada. Registro das evidências:

CENÁRIO	EVIDÊNCIA OBSERVADA
Resposta com dados da usuária	“Hoje é o dia 37 do seu ciclo, e você está na fase lútea” — valores coincidentes com os registros do banco.
<i>Streaming</i> progressivo	Texto exibido em fragmentos, sem espera pela resposta completa.
Persistência do diálogo	Par de mensagens gravado; título da conversa derivado da primeira mensagem.
Isolamento entre conversas	Conversa recém-criada apresenta histórico vazio.
Memória entre conversas	Em nova conversa, a assistente respondeu “Sim, você me contou que tem endometriose”.
Extração de memória	Após a correção do <i>prompt</i> : gravados “Tem endometriose”, “Costuma ter cólica forte no primeiro dia” e “Usa DIU de cobre”; descartado o dia do ciclo.
Requisição sem sessão	401 — “Não autorizado.”
Conversa de outra usuária	404 — “Conversa não encontrada.”
Recuperação de sessão órfã	Redirecionamento ao <i>login</i> , sem falha de execução.
Rotas autenticadas	As vinte telas do painel responderam 200.

Lacuna reconhecida. Não há, nesta versão, suíte de testes automatizados. As verificações acima foram conduzidas manualmente e por inspeção instrumentada do DOM e do banco. Recomenda-se, como prioridade técnica: testes unitários das funções puras de `lib/utls.ts` — `predictCycle` , `cycleStats` , `pregnancyProgress` e `currentStreak` concentram as regras de negócio e são determinísticas — seguidos de testes de ponta a ponta dos fluxos de autenticação e de conversa.

12. Implantação e operação

12.1 Ambiente local

```
npm install           # instala dependências
cp .env.example .env  # configura variáveis
npx auth secret       # gera AUTH_SECRET
npm run db:push       # aplica o schema ao banco
npm run db:seed       # popula dados de demonstração
npm run dev           # inicia em http://localhost:3000
```

12.2 Scripts disponíveis

SCRIPT	FUNÇÃO
npm run dev	Servidor de desenvolvimento com <i>hot reload</i> .
npm run build	prisma generate seguido do <i>build</i> de produção.
npm start	Servidor de produção.
npm run typecheck	Verificação de tipos sem emissão.
npm run lint	Análise estática.
npm run db:push	Sincroniza o <i>schema</i> com o banco.
npm run db:seed	Insere a usuária de demonstração e registros de exemplo.
npm run db:studio	Interface de inspeção do banco.
npm run db:reset	Recria o banco e reexecuta o <i>seed</i> .

12.3 Migração a produção

1. Alterar `provider` para `postgresql` em `schema.prisma` e apontar `DATABASE_URL` à instância gerenciada. A modelagem não emprega recursos exclusivos do SQLite, dispensando conversão de tipos.
2. Substituir `db push` por migrações versionadas (`prisma migrate deploy`).
3. Provisionar as variáveis de ambiente na plataforma de hospedagem, com `AUTH_SECRET` inédito.
4. Implantar em plataforma compatível com Next.js 15 (Vercel ou contêiner Node).
5. Habilitar limitação de taxa nas rotas `/api/auth` e `/api/chat` .
6. Instrumentar monitoramento de erros e observabilidade de custo da API da OpenAI.

Nota operacional (ambiente Windows). O *build* falha com `EPERM` ao renomear `query_engine-windows.dll.node` quando o servidor de desenvolvimento está em execução, pois o arquivo permanece bloqueado. Encerrar o processo `dev` antes de executar `npm run build` .

13. Limitações conhecidas e evolução

13.1 Limitações da versão 1.0

ITEM	SITUAÇÃO E IMPLICAÇÃO
PIN de bloqueio	O PIN é definido e armazenado como <i>hash</i> , porém a tela que o solicita na abertura ainda não existe — o controle é, na prática, inoperante.
Recuperação de senha	Não implementada. A perda da senha implica perda de acesso à conta.
Verificação de e-mail	Ausente. Endereços não são confirmados no cadastro.
Insights	Heurísticos e determinísticos; não empregam modelo de linguagem nem aprendizado estatístico.
Dados locais	Serviços públicos, campanhas e pontos de distribuição são orientativos; endereços e horários oficiais ainda não integrados.
Testes automatizados	Inexistentes (ver seção 11.2).
Latência da memória	A extração ocorre antes do encerramento do <i>stream</i> , adicionando 1 a 2 segundos ao final da requisição. Opção deliberada por confiabilidade da gravação; a resposta já se encontra visível.
Notificações	Lembretes são exibidos apenas no aplicativo; não há envio <i>push</i> .
Internacionalização	Interface exclusivamente em português do Brasil, sem camada de tradução.

13.2 Evolução planejada

HORIZONTE	INICIATIVAS
Curto	Tela de bloqueio por PIN; recuperação de senha; testes unitários das funções de domínio; gráficos de séries temporais; modo escuro.
Médio	Migração a PostgreSQL e implantação; notificações <i>push</i> e PWA instalável; edição de registros; contador de contrações e de movimentos fetais; módulo pós-parto.
Longo	Modo anônimo; compartilhamento com acompanhante; integração de dados oficiais da Secretaria Municipal de Saúde; assinatura Premium; <i>marketplace</i> de parceiros; painel de indicadores agregados para gestão pública.

14. Glossário

TERMO	DEFINIÇÃO
App Router	Sistema de roteamento do Next.js baseado na estrutura de diretórios, com renderização no servidor por padrão.
bcrypt	Função de derivação de chave para senhas, com fator de custo ajustável.
DPP	Data provável do parto, estimada em 280 dias a partir da DUM.
DUM	Data da última menstruação; referência do cálculo gestacional.
Edge Runtime	Ambiente de execução restrito, próximo ao usuário, sem APIs completas do Node.
Fase lútea	Etapa do ciclo entre a ovulação e a menstruação seguinte, de duração relativamente estável.
Janela fértil	Intervalo de maior probabilidade de concepção, estimado entre cinco dias antes e um dia após a ovulação.
JWT	<i>JSON Web Token</i> ; formato de credencial assinada empregado na sessão.
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei 13.709/2018).
Prompt de sistema	Instrução inicial que estabelece papel, tom e limites do modelo de linguagem.
Server Action	Função executada no servidor, invocável diretamente por formulários, sem endpoint explícito.
SSE	<i>Server-Sent Events</i> ; protocolo de envio progressivo empregado no <i>streaming</i> da assistente.
Streaming	Entrega incremental da resposta, exibida à medida que é gerada.
UBS	Unidade Básica de Saúde, porta de entrada da atenção primária no SUS.
Zod	Biblioteca de validação de esquemas que também origina os tipos TypeScript correspondentes.